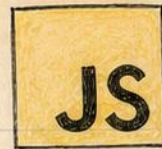




JAVASCRIPT — FROM ZERO TO HERO



Ch. 1 — What is JavaScript? | Study Notes 1/10

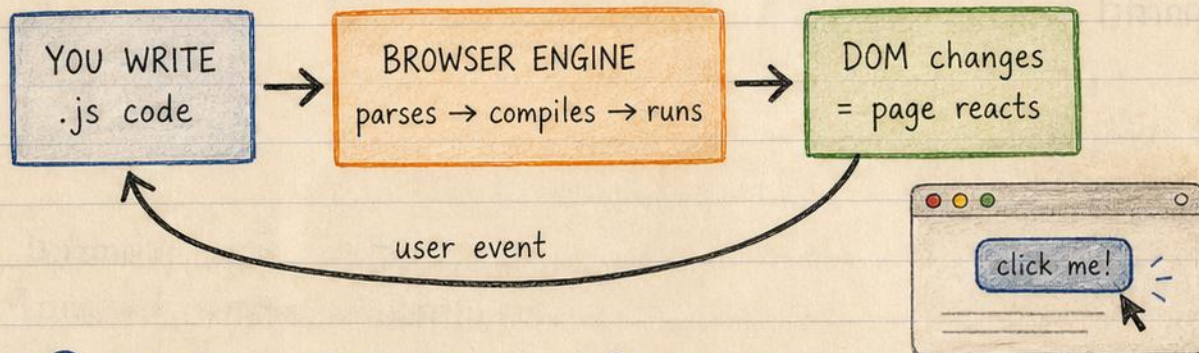
Date: _____

① CORE DEFINITION

JS = a high-level, interpreted language that runs in the browser & on servers (Node). It makes pages INTERACTIVE.

line-by-line,
no build step

② BASIC OVERVIEW



③ SYNTAX — FIRST CODE

```
let name = "Madurai";  
console.log("Hello, " + name);  
// → Hello, Madurai
```

④ PRACTICAL USE CASES

-  • form validation
-  2:1 • live scores / chat
-  • lazy loading images

HTML = skeleton, CSS = style,
JS = muscles

⑤ KEY POINTS

Runs where	browser + Node
Typing	dynamic, weakly typed
Paradigm	multi: OOP + functional
File	.js, loaded via <script>



Next slide → variables & data types

VARIABLES & DATA TYPES

Ch. 2-3

Notes 2/10

VAR vs LET vs CONST

var

- function-scoped
- can leak
- avoid

let

- block-scoped
- reassign OK

5

→ 7

const

- block-scoped
- binding FROZEN



Declaration = naming a box;
assignment = putting value in it.

```
let age = 18;      // reassignable
age = 19;
const PI = 3.14159;
// PI = 3; ✗ TypeError
var lang = "JS";   // hoisted → visible before line
```

USE CASES



user settings
→ **const**

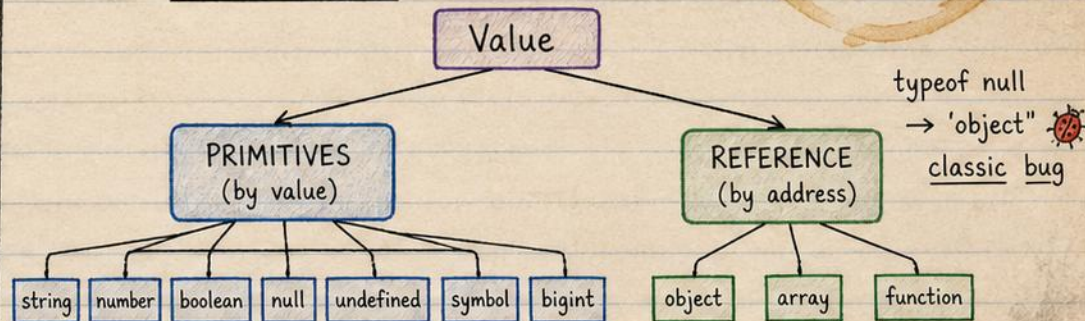


loop counter
→ **let**

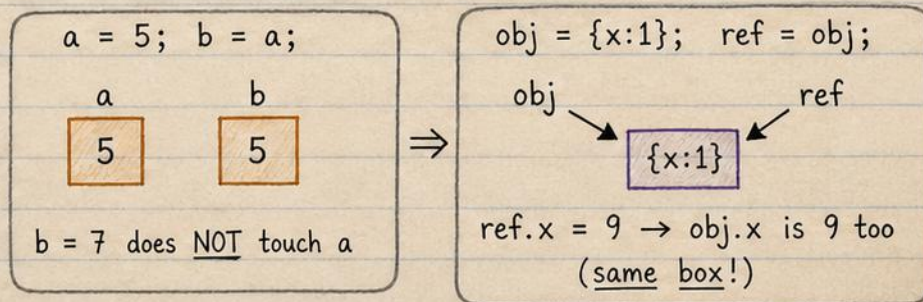


config table
→ frozen object

TYPE TREE



COPYING DIAGRAM



KEY POINTS

Declare	let / const , never var
Undeclared read	ReferenceError
Declared no value	undefined
Compare safely	=== (no coercion)

OPERATORS, CONDITIONALS & LOOPS

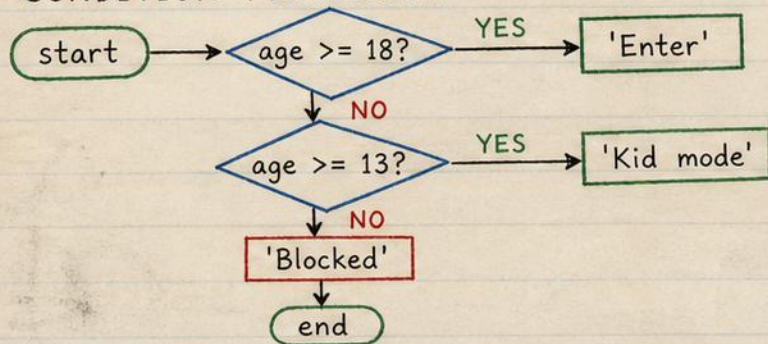
Ch. 4 | Notes 3/10

OPERATOR FAMILIES

→ arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code>
→ comparison	<code>></code> <code><</code> <code>>=</code> <code>===</code>
→ logical	<code>&&</code> <code> </code> <code>!</code>
→ assignment	<code>=</code> <code>+=</code> <code>??=</code>
→ special	<code>typeof</code> <code>?</code> <code>:</code> <code>?</code>

`==` coerces types, `===` does NOT —
always use `===`

CONDITION FLOWCHART



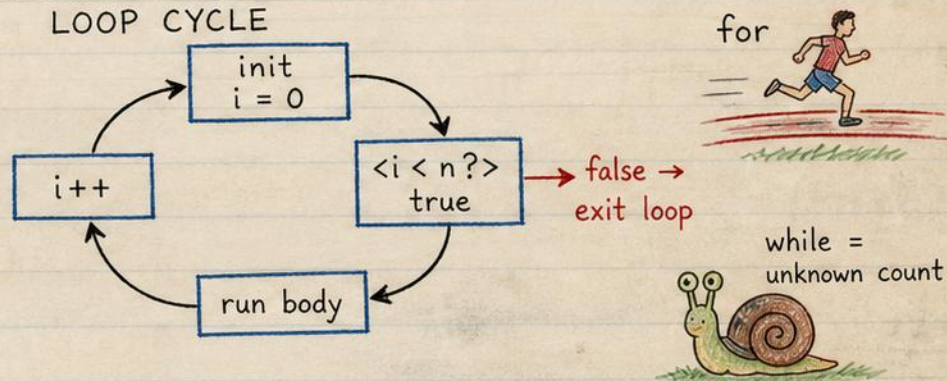
```
for (let i = 1; i <= 3; i++) {  
  console.log("Row " + i);  
}  
let n = 0;  
while (n < 2) n++;  
const msg = n > 1 ? "many" : "few";
```

TRUTH TABLE

A	B	A && B
T	T	T
T	F	F
F	—	F

`a ?? b` → only null / undefined trigger the fallback

LOOP CYCLE



USE CASES

- login gate
- product list loops
- retry until success

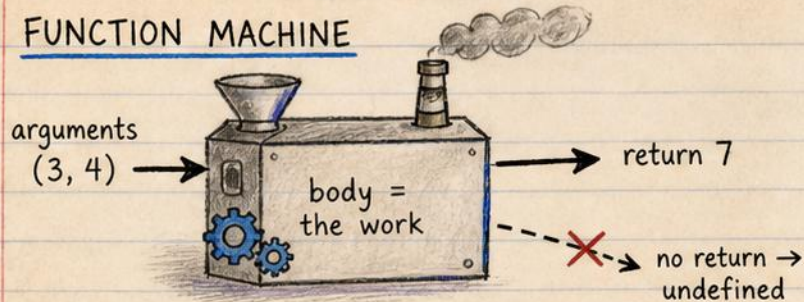
KEY POINTS

break / continue	stop / skip
for..of	values
for..in	object keys

FUNCTIONS, SCOPE & CLOSURES

Ch. 5 | Notes 4/10

FUNCTION MACHINE



DECLARATION

function add(a,b){ return a+b; } ↑ hoisted

EXPRESSION

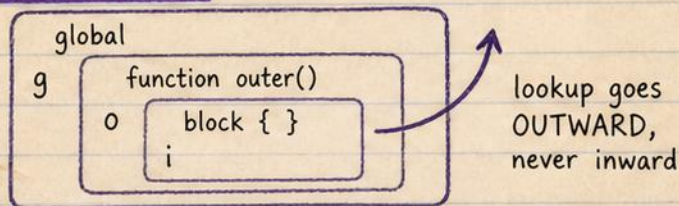
const add = function(a,b){ return a+b; }

ARROW

const add = (a,b) => a+b; ← implicit return

arrow has NO own this

SCOPE CHAIN



inner can read outer — outer CANNOT read inner

```
function makeCounter() {  
  let count = 0;           // private!  
  return () => ++count;     // closure  
}  
const c = makeCounter();  
c(); c(); // → 1, then 2  
[1,2,3].map(n => n * 10); // callback = fn as value
```

USE CASES



- map/filter callbacks
- private counters & timers
- event handlers
- debounce

CLOSURE

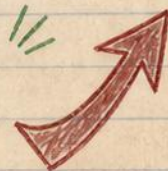
makeCounter() returns



the returned fn remembers its birthplace variables.

KEY POINTS

return	stops fn, sends value
default	(a = 5) params
rest	(...nums) → array
this	normal fn: caller; arrow: lexical
HOF	fn taking/returning fn



ARRAYS & OBJECTS (+ stack vs heap)

Ch. 6 | Notes 5/10

ARRAY = numbered boxes

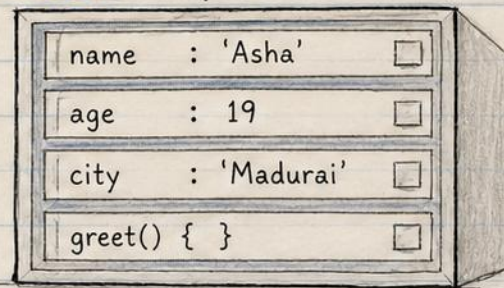


index starts at 0

$.length - 1 = 6$

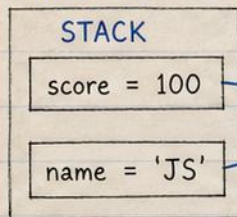
push/pop → end (fast) • shift/unshift → front (slow, re-shifts all)

OBJECT = keyed drawer



dot obj.name |
bracket obj['city']
(needed for
dynamic keys)

STACK vs HEAP



stores only the
ADDRESS (reference)

copy a primitive = new value • copy a reference = same object

map → [1, 2, 3] ⇒ [2, 4, 6] (same length, transformed)
filter → [1, 2, 3] ⇒ [2, 3] (funnel narrows, keeps true)
reduce → [1, 2, 3] ⇒ 6 (funnel crushing into ONE value)

```
const cart = [
  { item: "notebook", qty: 3, price: 40 },
  { item: "pen", qty: 5, price: 10 },
];
const total = cart
  .filter(p => p.qty > 0)
  .reduce((s, p) => s + p.qty * p.price, 0);
// 170
const { item, qty } = cart[0]; // destructure
```

USE CASES

- shopping cart totals
- search filters
- grouping records
- JSON from APIs

KEY POINTS

array order
| kept

sort mutates!
| slice().sort()

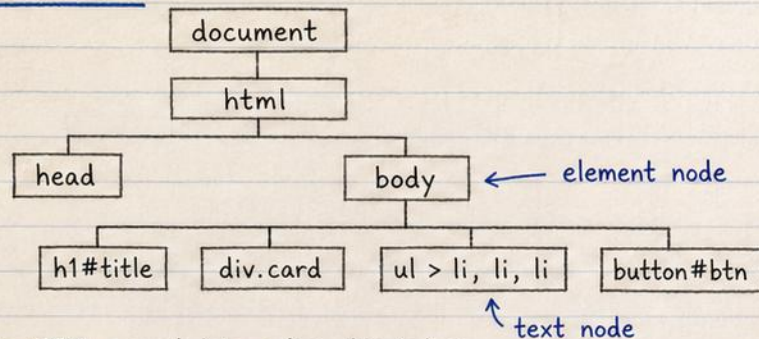
find vs filter
| one vs many

immutability
| [...arr, x]

THE DOM — SELECT, CHANGE, REACT

Ch. 7 | Notes 6/10

DOM TREE



the HTML parsed into a live object tree
JS can poke

```
const btn = document.querySelector("#btn");
btn.textContent = "Clicked!";
btn.classList.toggle("active");
```

```
const li = document.createElement("li");
li.innerText = "new task";
document.querySelector("ul").append(li);
```

```
btn.addEventListener("click", e => {
  console.log(e.target.id, e.type);
});
```

innerHTML vs textContent

rendered

as literal text

```
...
<h1>Hello</h1>
<p>Welcome!</p>
```

```
...
<h1>Hello</h1>
<p>Welcome!</p>
```

user text? use `textContent` (XSS safe)

live vs stored



`querySelectorAll`
= snapshot,
not live

GET / SET

JS object
(element)

`querySelector` →
read/write
`.textContent`, `.style`,
`.classList`

on-screen
pixels



EVENT FLOW

1 capture



delegation = 1 listener on parent,
read `e.target`

USE CASES

- to-do list add/remove
- dark-mode toggle
- navbar scroll-spy
- modal popups

KEY POINTS

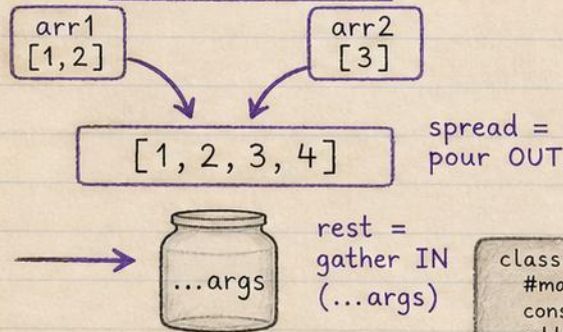
select	<code>querySelector(All)</code>
style	<code>classList</code> not string hack
on	<code>addEventListener</code> (many fns)
off	<code>removeEventListener</code> / <code>AbortController</code>

ES6+ — MODERN SYNTAX TOOLKIT

Ch. 8 | Notes 7/10

	ES5 (old)	THEN vs NOW	ES6+ (now)
1.	<code>var i = 0</code>	→	<code>let / const</code>
2.	<code>'Hi ' + n + '!</code>	→	<code>'Hi \${n}!'</code>
3.	<code>function(x){ return x*2 }</code>	→	<code>x => x * 2</code>
4.	<code>var a = o.a; var b = o.b;</code>	→	<code>const {a, b} = o;</code>

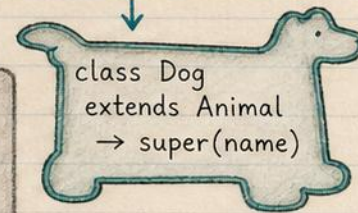
SPREAD & REST



CLASS

```
class Animal
  constructor(name)
  speak()
```

classes are
sugar over
prototypes



```
class Student {
  #marks = 0; // private field
  constructor(name) { this.name = name; }
  add(m) { this.#marks += m; return this; }
  get avg() { return this.#marks / 3; }
}

const s = new Student("Asha");
s.add(90).add(80).add(70);
console.log(s.avg, s?.city?.pin ?? "n/a");
```

EXTRA TOOLS

- Map/Set = keyed & unique
- Symbol = unique key
- Proxy = trap on object
- Modules: import / export

USE CASES

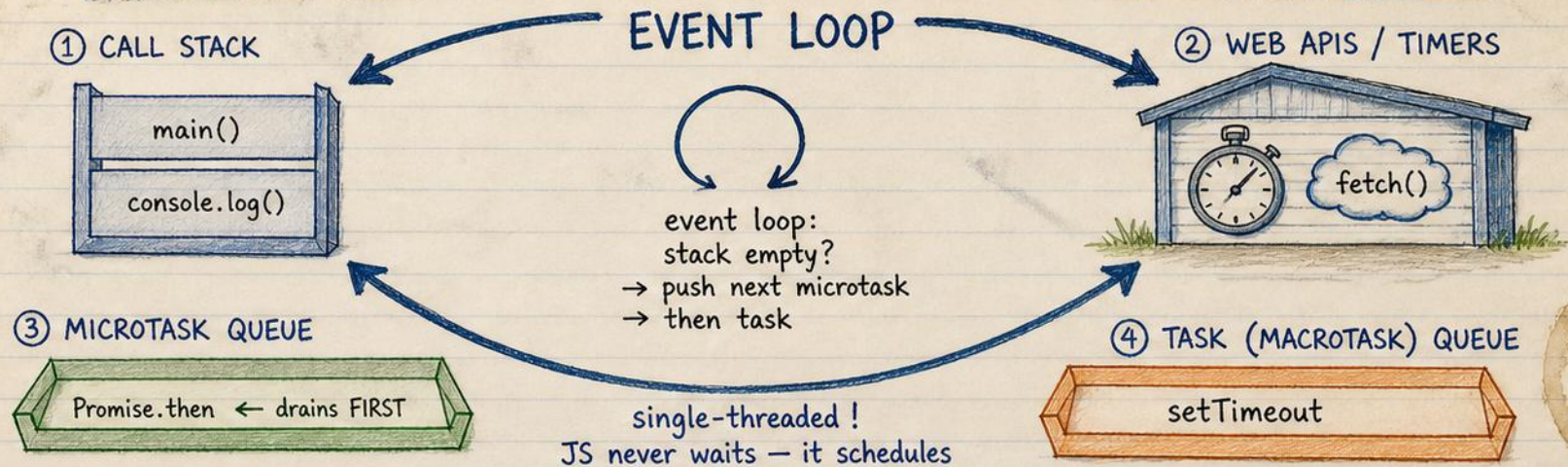
- component state spread
- cloning without mutation
- optional chaining on API data

KEY POINTS

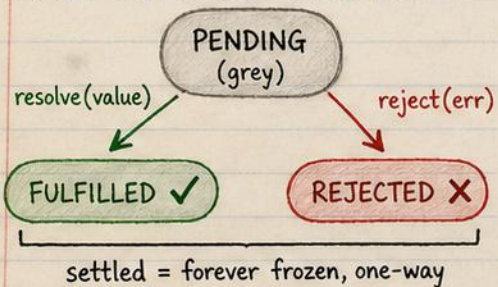
template literal	multi-line + <code>\${expr}</code>
default params	<code>fn(a = 1)</code>
shorthand	<code>{a, b, c}</code>
for..of	iterates values

ASYNC JS - EVENT LOOP, PROMISES, AWAIT

Ch. 9 | Notes 8/10



PROMISE STATE MACHINE



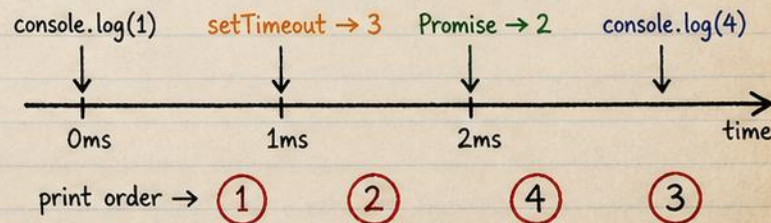
USE CASES

- API data + loading spinner
- infinite scroll
- retry failed request
- parallel image loads

```
async function getUser(id) {
  try {
    const res = await fetch("/api/user/" + id);
    if (!res.ok) throw new Error(res.status);
    return await res.json();
  } catch (e) {
    console.error("failed", e);
  } finally {
    loader.hidden = true;
  }
}

const all = await Promise.all([a, b, c]);
```

TIMELINE EXAMPLE (print order)



KEY POINTS

callback hell	nested, hard to catch
Promise	chainable, .catch
async/await	sync-looking, needs try/catch
Promise.all	all-or-nothing
Promise.allSettled	always resolves



ADVANCED PATTERNS & TRICKS

Ch. 10 | Notes 9/10

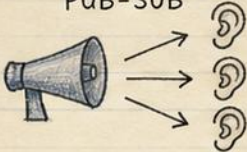
PATTERN CARDS

① **MODULE** — reveal API



hide internals,
expose 3 fns

② **OBSERVER / PUB-SUB**



emit('click') →
many listeners

③ **SINGLETON**



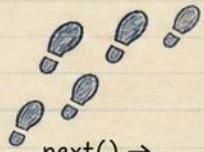
one instance
ever (config)

④ **FACTORY**



fn that returns
configured objects

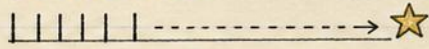
⑤ **ITERATOR**



next() →
{value, done}
• for...of



DEBOUNCE vs THROTTLE

DEBOUNCE:  fires only after the pause

THROTTLE:  max one fire per gap

search box → debounce • scroll/resize → throttle

IMMUTABLE UPDATE

~~{user,
age:19}~~

spread →

{user,
age:20}

no mutation → predictable state

 (React: setState with new object)

```
// Module pattern
const store = (() => {
  let n = 0;
  return { inc: () => ++n, get: () => n };
})();
```

```
// Debounce
function debounce(fn, ms = 300) {
  let t;
  return (...a) => { clearTimeout(t);
    t = setTimeout(() => fn(...a), ms); };
}
```



ALSO GOOD TO KNOW

- optional chaining ?.
- nullish ??
- structural sharing
- WeakMap for caches
- GC = automatic, avoid leaked globals



KEY POINTS

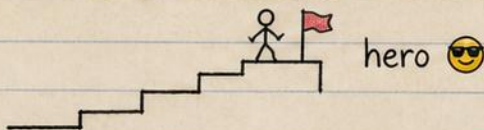
encapsulate	closure / #
decouple	pub-sub
performance	memoize, debounce
state	copy, don't mutate
one source	singleton config



QUICK REVISION MAP

Summary | Notes 10/10

LEARNING ROADMAP



ONE-LINE RECAP

let / const	block scope
===	no coercion
function	input → work → return
closure	remembers its scope
map / filter / reduce	transform / keep / crush
DOM	select → change → listen
Promise	pending → settled

PITFALLS

- ✗ var leaking to global
- ✗ == surprising you
- ✗ mutating a shared object
- ✗ forgetting await
- ✗ infinite loop = frozen tab

PRACTICE

- Q1 `const o = {n:1}; const p = o; p.n = 9; console.log(o.n);` → ?
- Q2 Which logs LAST? A) `Promise.then` B) `console.log` C) `setTimeout(...,0)`
- Q3 Write `unique(arr)` using `new Set()`.

A1 → 9 (one box, two names)
A2 → A
A3 → [...new Set(a)]

✓ revise before mock test

PRACTICE QUESTIONS — MCQ & CODE

Exercises | Notes 11/11

① Q1 MCQ

```
let x = 1;  
function f(){ let x = 2; }  
f();  
console.log(x);
```

a) 1 b) 2 c) error

② Q2 MCQ

typeof null is...

a) null b) 'object' c) 'number'

③ Q3 MCQ

[1, 2, 3].map(n => n * 2) gives

a) [2, 4, 6] b) 12 c) [1, 2, 3, 2, 4, 6]

④ Q4 WRITE

Add a click listener that toggles .active on #btn — three ways to attach and one to remove

- ☐ onclick ✓
- ☐ addEventListener ✓
- ☐ delegate ✓

⑤ Q5 FIX THE BUG

```
async function load(){  
  const r = await fetch(url)  
  const d = r.json()  
  return d.length  
}
```

missing await

res.ok not checked

ANSWERS

Q1 a (block scope) • Q2 b (classic bug) • Q3 a •

Q4 AbortController/off • Q5 await r.json() + if(!r.ok) throw



0 → hero!